Autonomous Humanoid Football-Kicking Robot

AI for Robotics Course

📌 Overview

Design and build a **low-cost autonomous humanoid robotic system** that detects a football using a camera, computes leg joint angles through forward and inverse kinematics, and executes a kicking motion with servo control.

The system must be built using **ROS 2**, following a modular architecture with perception, planning, and control nodes.

PART 1 — Mechanical Design

Design a low-cost, simplified bipedal lower body with two leg segments per kicking leg, each driven by a servo motor. The robot must stand stably on a flat surface using a fixed base.

PART 2 — Robot Kinematics

Forward Kinematics (FK) answers the question: given joint angles θ_1 and θ_2, where is the foot? Deriving FK is the essential foundation; inverse kinematics simply inverts this model. FK is also used at runtime to validate IK output and catch errors.

2-DOF Planar Leg — FK Equations

$x = L_1 \cdot \cos(\theta_1) + L_2 \cdot \cos(\theta_1 + \theta_2)$

$y = L_1 \cdot \sin(\theta_1) + L_2 \cdot \sin(\theta_1 + \theta_2)$

Where: L_1 = thigh length, L_2 = shin length, θ_1 = hip angle, θ_2 = knee angle relative to thigh

Inverse Kinematics (IK) Given a desired foot target position (x, y) received from the vision system, the IK solver computes the required hip angle θ_1 and knee angle θ_2. These are then mapped to servo PWM values, and the actuators are commanded accordingly.

• Use geometric methods (law of cosines) to solve for joint angles from link lengths L_1, L_2, and target position

• The computed angles should be published as **ROS** topics and consumed by the actuator control node.

PART 3 — Computer Vision: Ball Detection

Use a camera module (ESP32-CAM, USB webcam) to detect the position of a football in real time and estimate its position relative to the humanoid robot.

Publish ball coordinates (e.g., `/ball_position`). And the MCU would use these coordinates as the IK target position.

PART 4 — Autonomous Behavior

- Design a modular system using **ROS 2** and implement the humanoid behavior.
- Visualize humanoid motion in RViz.
- Use the TF tree for coordinate transformations.
- The system must operate fully **autonomously**.

PART 5 — Report Structure

The written report is a key deliverable. Follow this exact structure:

Chapter 1: Introduction

- Historical background
- Problem statement
- Aims and objectives of the project
- Report agenda

Chapter 2: Literature Review

A thorough survey of prior work. Cover:

- Design & mechanical
- Electronics & software
- Problems & limitations of prior work
-

Chapter 3: Proposed System

- System overview
- Improvements over existing humanoid approaches
- System Block Diagrams (Hardware and Software)

Chapter 4: Implementation

- Mechanical design
- ROS nodes and communication
- State Machine
- Implemntation
- Bill of Materials (BoM)

Chapter 5: Results

- FK accuracy: measured vs. computed foot position
- IK accuracy: target vs. achieved angle
- Vision latency measurement and detection accuracy

Chapter 6: Conclusion

Challenges (NOT Required, but if you did, it would be a bonus)

1. MATLAB / Simulink simulation

Before or alongside hardware implementation, build a MATLAB simulation of the 2-DOF leg kinematics. Use Simulink to model the servo dynamics.

Recommended resource: Humanoid Robot MATLAB tutorial.

2. Wireless dashboard

Build a simple web dashboard (served from an ESP32 or a companion device) that visualizes live telemetry: ball coordinates, joint angles, and state-machine status. Display using a browser over Wi-Fi.

Adding any other enhancement will be considered for bonus marks.

Full Deliverables

Students must submit a complete project package that includes the following:

1. Source Code (ROS packages)
2. Report (PDF)
3. Presentation
4. Marketing poster
5. Demo video
6. Any Additional Materials (Simulations, Dashboard, Screenshots)

Academic Integrity & Resources

The use of Large Language Models (LLMs) such as ChatGPT, Claude, and Gemini to generate code for this project is **strictly prohibited**. Submitting code generated by an LLM will be flagged for similarity with other submissions, as LLMs often produce similar solutions, which will result in a penalty for plagiarism. Students are permitted to use LLMs only to learn new concepts or search for resources.